

INSTITUTO FEDERAL DO TRIÂNGULO MINEIRO

Campus Uberlândia Centro

Estruturas de Dados — 2026/1

Curso Superior de Tecnologia em Sistemas para Internet

PROJETO INTEGRADOR

EduMetrics API

Sistema de Acompanhamento de Desempenho Acadêmico
com Análise de Performance e Benchmarking

Entrega Final | 40 pontos



Ficha de Identificação do Grupo

Grupo: _____

Integrante 1: _____

Integrante 2: _____

Repositorio GitHub: <https://github.com/>_____

Data de entrega: ____ / ____ / 2026

1. Visão Geral do Projeto

O EduMetrics API é um sistema RESTful de acompanhamento de desempenho acadêmico de alunos do IFTM. O projeto é um ambiente de integração prática de todas as estruturas de dados e técnicas de performance estudadas ao longo do semestre. Mais do que um sistema que funciona, o objetivo é um sistema que funciona bem — com cada decisão de estrutura de dados justificada por análise de complexidade e cada otimização comprovada por benchmark JMH.

O EduMetrics resolve quatro problemas reais de performance:

Problema de produção	Estrutura aplicada	Resultado esperado
Latência alta em consultas de aluno	Redis + @Cacheable	De 500 ms para < 5 ms (cache hit)
Busca de disciplina é lenta (varredura linear)	Trie em memória	$O(\text{prefixo})$ independente do n
Relatórios bloqueiam endpoints síncronos	RabbitMQ + @RabbitListener	Endpoint retorna em < 20 ms
Sem proteção contra abuso de API	Sliding Window Rate Limiter	100 req/min por IP, preciso

1.1 Conexão com as Unidades do Semestre

Unidade	Estruturas estudadas	Onde aparecem no EduMetrics
I — Fundamentos	ArrayList, LinkedList, Pilha, Fila	ArrayList em repositórios; ArrayDeque no Rate Limiter
II — Árvores/Heaps	BST, AVL, Heap, PriorityQueue	PriorityQueue para ranking Top-K de alunos
III — Hash Tables	TabelaHash<K,V>, HashMap, HashSet	HashMap para índice matrícula→aluno; HashSet para deduplicação
IV — Web/Performance	Redis, RabbitMQ, Trie, LRU, JMH	Todo o sistema — cache, mensageria, autocompletar, benchmark



Princípio central do projeto

Não basta implementar. É preciso medir.

Cada estrutura de dados escolhida deve ser justificada com complexidade teórica e comprovada com resultado empírico do JMH. A frase "é mais rápido" sem dados não tem valor técnico. Neste projeto, dados substituem opiniões.

2. Especificação Técnica

2.1 Stack e Dependências

Tecnologia	Versão	Papel no projeto
Java	JDK 21+	Linguagem principal; records, pattern matching, sealed classes
Spring Boot	3.3.x	Framework web, IoC, auto-configuração
Spring Data JPA	3.3.x	Persistência + geração de índices B+ via @Index
Spring Cache	3.3.x	Abstração @Cacheable/@CachePut/@CacheEvict
Spring Data Redis	3.3.x	RedisTemplate e integração com Spring Cache
Spring AMQP	3.3.x	Producer RabbitTemplate e Consumer @RabbitListener
H2 Database	última	Banco em memória para desenvolvimento e testes
Redis	7.x (Docker)	Cache distribuído com TTL
RabbitMQ	3.x (Docker)	Message broker para relatórios assíncronos
JMH	1.37	Benchmarking de microbenchmarks
JUnit 5 + Mockito	5.10.x	Suíte de testes; cobertura >= 80% (JaCoCo)
Maven	3.9+	Build e gestão de dependências

2.2 Infraestrutura Docker

Crie o arquivo docker-compose.yml na raiz do projeto:

```
version: "3.9"

services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    healthcheck:
```

```

    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5

rabbitmq:
  image: rabbitmq:3-management-alpine
  ports:
    - "5672:5672" # AMQP
    - "15672:15672" # painel de administração (guest/guest)
  environment:
    RABBITMQ_DEFAULT_USER: guest
    RABBITMQ_DEFAULT_PASS: guest
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

# Iniciar: docker compose up -d
# Parar: docker compose down

```

2.3 Configuração application.properties

```

# — Banco de dados (H2 em memoria) —————
spring.datasource.url=jdbc:h2:mem:edumetrics;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=false # true apenas para depurar N+1

# — Redis —————
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.data.redis.timeout=2000ms

# — Cache —————
spring.cache.type=redis
spring.cache.redis.time-to-live=300000 # 5 minutos em milissegundos
spring.cache.redis.cache-null-values=false

# — RabbitMQ —————
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

# — Logs —————
logging.level.br.edu.iftm=DEBUG
logging.level.org.springframework.cache=TRACE # ver cache hits/misses

```

2.4 Estrutura de Pacotes

```

br.edu.iftm.edumetrics
├── EduMetricsApplication.java

```

```

└─ config/
  └─ CacheConfig.java           // RedisCacheManager, TTL, serializer
  └─ RabbitMQConfig.java        // Exchange, Queue, Binding, DLQ
  └─ DataLoader.java            // @PostConstruct: popula Trie e dados
iniciais
└─ controller/
  └─ AlunoController.java        // CRUD + autocompletar
  └─ DesempenhoController.java   // notas, medias, ranking
  └─ RelatorioController.java    // POST assíncrono → RabbitMQ
  └─ AdminController.java        // cache stats, health
└─ service/
  └─ AlunoService.java           // @Cacheable, @CacheEvict; indice HashMap
  └─ DesempenhoService.java      // calculos de media; cache de resultados
  └─ RankingService.java         // PriorityQueue para Top-K
  └─ AutocompletarService.java   // Trie carregada no startup
└─ messaging/
  └─ RelatorioProducer.java       // RabbitTemplate.convertAndSend
  └─ RelatorioConsumer.java      // @RabbitListener + DLQ handler
└─ domain/
  └─ Aluno.java                  // @Entity @Table(indexes={...})
  └─ Disciplina.java
  └─ Desempenho.java             // @ManyToOne(fetch=LAZY)
  └─ dto/
    └─ AlunoDTO.java             // record - imutavel, ideal para cache
    └─ DesempenhoDTO.java
    └─ RankingItemDTO.java
    └─ EventoRelatorio.java      // mensagem AMQP
└─ repository/
  └─ AlunoRepository.java         // JpaRepository + @EntityGraph
  └─ DesempenhoRepository.java    // queries com JOIN FETCH
└─ estruturas/
  └─ LRUCache.java               // extends LinkedHashMap<K,V>
  └─ Trie.java                   // arvore de prefixos
  └─ RateLimiter.java            // Sliding Window com Deque<Long>
└─ security/
  └─ RateLimitFilter.java        // OncePerRequestFilter
└─ benchmark/
  └─ EduMetricsBenchmark.java     // suite JMH

```

3. Modelo de Dados

3.1 Entidades JPA

```
// — Aluno —————
@Entity
@Table(name = "alunos", indexes = {
    // Cria árvore B+ na coluna matricula – busca O(log n) no BD
    // (complementa o HashMap em memória para persistência)
    @Index(name = "idx_aluno_matricula", columnList = "matricula", unique =
true),
    @Index(name = "idx_aluno_nome",      columnList = "nome")
})
public class Aluno {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 12)
    private String matricula;      // chave do HashMap em memória

    @Column(nullable = false, length = 120)
    private String nome;          // indexado na Trie

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(nullable = false)
    private String curso;

    @Column(nullable = false)
    private Integer periodo;

    @OneToMany(mappedBy = "aluno", fetch = FetchType.LAZY,
        cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Desempenho> desempenhos = new ArrayList<>();

    // construtores, getters, equals/hashCode por id
}

// — Disciplina —————
@Entity
@Table(name = "disciplinas", indexes = {
    @Index(name = "idx_disc_codigo", columnList = "codigo", unique = true)
})
public class Disciplina {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 10)
    private String codigo;        // ex: "ED01"

    @Column(nullable = false, length = 100)
    private String nome;          // inserido na Trie no startup

    @Column(nullable = false)
```

```

        private Integer creditos;
    }

    // — Desempenho —————
    @Entity
    @Table(name = "desempenhos", indexes = {
        @Index(name = "idx_desemp_aluno", columnList = "aluno_id"),
        @Index(name = "idx_desemp_aluno_disc", columnList = "aluno_id,
disciplina_id")
    })
    public class Desempenho {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        @ManyToOne(fetch = FetchType.LAZY)
        @JoinColumn(name = "aluno_id", nullable = false)
        private Aluno aluno;

        @ManyToOne(fetch = FetchType.LAZY)
        @JoinColumn(name = "disciplina_id", nullable = false)
        private Disciplina disciplina;

        @Column(precision = 5, scale = 2)
        private BigDecimal nota1;

        @Column(precision = 5, scale = 2)
        private BigDecimal nota2;

        @Column(precision = 5, scale = 2)
        private BigDecimal notaFinal; // calculado: (nota1 + nota2) / 2

        @Column(nullable = false, length = 7)
        private String semestre; // ex: "2026/1"
    }

```

3.2 DTOs (Records Java 21)

```

// Records são imutáveis – ideais para cache (sem risco de mutação após
armazenar)

public record AlunoDTO(
    Long id, String matricula, String nome,
    String email, String curso, Integer periodo
) {}

public record DesempenhoDTO(
    String disciplina, BigDecimal nota1,
    BigDecimal nota2, BigDecimal notaFinal, String semestre
) {}

public record RankingItemDTO(
    int posicao, String nome, String matricula,
    double mediaGeral, int disciplinasConcluidas
) implements Comparable<RankingItemDTO> {
    @Override
    public int compareTo(RankingItemDTO outro) {

```

```
        // Ordem crescente de media - necessário para o min-heap do
PriorityQueue
        return Double.compare(this.mediaGeral, outro.mediaGeral);
    }
}

// Mensagem enviada ao RabbitMQ - deve ser serializável (Jackson)
public record EventoRelatorio(
    String correlationId,    // UUID para rastrear o relatorio
    Long alunoId,
    String tipo,             // "BOLETIM", "HISTORICO", "RANKING"
    String semestre,
    Instant solicitadoEm
) implements Serializable {}
```


4. Implementação das Estruturas Customizadas

As três estruturas abaixo DEVEM ser implementadas manualmente pelo grupo — não é permitido substituir por bibliotecas externas. Cada uma tem especificação de interface, código-base e critérios de teste.

4.1 LRUCache<K, V>

Complexidade obrigatória: get O(1) amortizado, put O(1) amortizado, evicção O(1).

```
/**
 * Cache LRU (Least Recently Used) baseado em LinkedHashMap com
 * accessOrder=true.
 * Quando a capacidade máxima é atingida, o elemento acessado há mais tempo é
 * removido.
 *
 * @param <K> tipo da chave — deve ser imutável (String, Long, record)
 * @param <V> tipo do valor
 */
public class LRUCache<K, V> extends LinkedHashMap<K, V> {

    private final int capacidade;

    /**
     * @param capacidade número máximo de entradas antes de iniciar evicção
     */
    public LRUCache(int capacidade) {
        // accessOrder = true: cada get() move o nó para o fim da lista
        // loadFactor = 0.75f: padrão JCF
        super(capacidade, 0.75f, /* accessOrder */ true);
        this.capacidade = capacidade;
    }

    /**
     * Invocado pelo LinkedHashMap após cada put().
     * Retorna true para remover a entrada mais antiga quando capacity é
     * excedida.
     */
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacidade;
    }

    /**
     * Fabrica thread-safe para uso em ambientes multi-thread (ex: requisições
     * HTTP).
     */
    public static <K, V> Map<K, V> create(int capacidade) {
        return Collections.synchronizedMap(new LRUCache<>(capacidade));
    }

    public int getCapacidade() { return capacidade; }
}

// Uso no AlunoService:
@Service
public class AlunoService {
```

```

// Cache local para busca por matrícula - 500 entradas
private final Map<String, AlunoDTO> cacheMatricula = LRUCache.create(500);

// Contadores para o endpoint /admin/cache/stats
private final AtomicLong hits = new AtomicLong();
private final AtomicLong misses = new AtomicLong();

public AlunoDTO buscarPorMatricula(String matricula) {
    AlunoDTO dto = cacheMatricula.get(matricula);
    if (dto != null) {
        hits.incrementAndGet();
        return dto;
    }
    misses.incrementAndGet();
    Aluno aluno = repository.findByMatricula(matricula)
        .orElseThrow(() -> new AlunoNaoEncontradoException(matricula));
    dto = toDTO(aluno);
    cacheMatricula.put(matricula, dto);
    return dto;
}

public Map<String, Object> cacheStats() {
    long h = hits.get(), m = misses.get(), total = h + m;
    return Map.of(
        "entradas", cacheMatricula.size(),
        "capacidade", ((LRUCache<?,?>)
            ((java.util.Collections.SynchronizedMap)
                cacheMatricula)).m.getCapacidade(), // via reflexão
        "hits", h,
        "misses", m,
        "hitRate", total == 0 ? 0.0 : (double) h / total
    );
}
}

```

4.2 Trie (Árvore de Prefixos)

Complexidade obrigatória: inserir $O(|\text{palavra}|)$, autocompletar $O(|\text{prefixo}| + k)$.

```

/**
 * Árvore de prefixos para autocompletar nomes de disciplinas.
 * Cada nó representa um caractere; caminhos da raiz até nos terminais
 * formam as palavras inseridas.
 *
 * Complexidade de busca:  $\Theta(|\text{prefixo}|)$  - independente do total de palavras.
 */
public class Trie {

    private static class No {
        final Map<Character, No> filhos = new HashMap<>();
        boolean fimDePalavra = false;
        String valorCompleto; // armazena o termo completo (com formatação
original)
    }

    private final No raiz = new No();
    private int totalPalavras = 0;

```

```

/**
 * Insere uma palavra na Trie.
 * Complexidade:  $\Theta(|palavra|)$ 
 */
public void inserir(String palavra) {
    if (palavra == null || palavra.isBlank()) return;
    No atual = raiz;
    // Indexamos em minúsculo para busca case-insensitive
    for (char c : palavra.toLowerCase().toCharArray()) {
        atual = atual.filhos.computeIfAbsent(c, k -> new No());
    }
    if (!atual.fimDePalavra) {
        atual.fimDePalavra = true;
        atual.valorCompleto = palavra; // preserva formatação original
        totalPalavras++;
    }
}

/**
 * Retorna até maxResultados termos que começam com o prefixo.
 * Complexidade:  $\Theta(|prefixo|)$  para a descida +  $O(k)$  para a coleta.
 *
 * @param prefixo string de busca (case-insensitive)
 * @param maxResultados limite superior de resultados
 * @return lista de termos correspondentes, em ordem de DFS
 */
public List<String> autocompletar(String prefixo, int maxResultados) {
    if (prefixo == null || prefixo.isBlank()) return List.of();
    No no = descerAte(prefixo.toLowerCase());
    if (no == null) return List.of();

    List<String> resultados = new ArrayList<>();
    coletarPalavras(no, resultados, maxResultados);
    return Collections.unmodifiableList(resultados);
}

private No descerAte(String prefixo) {
    No atual = raiz;
    for (char c : prefixo.toCharArray()) {
        atual = atual.filhos.get(c);
        if (atual == null) return null;
    }
    return atual;
}

private void coletarPalavras(No no, List<String> resultado, int max) {
    if (resultado.size() >= max) return;
    if (no.fimDePalavra) resultado.add(no.valorCompleto);
    for (No filho : no.filhos.values()) {
        coletarPalavras(filho, resultado, max);
    }
}

public int size() { return totalPalavras; }
public boolean isEmpty() { return totalPalavras == 0; }
}

```

```
// Uso no AutocompletarService:
@Service
public class AutocompletarService {

    private final Trie trie = new Trie();

    @Autowired
    private DisciplinaRepository disciplinaRepo;

    @PostConstruct // executa uma vez no startup da aplicação
    public void carregarDisciplinas() {
        disciplinaRepo.findAll().forEach(d -> trie.inserir(d.getNome()));
        log.info("Trie carregada com {} disciplinas", trie.size());
    }

    public List<String> sugerir(String prefixo) {
        return trie.autocompletar(prefixo, 10);
    }

    // Chamado quando uma nova disciplina e cadastrada
    public void indexar(String nomeDisciplina) {
        trie.inserir(nomeDisciplina);
    }
}
```

4.3 RateLimiter (Sliding Window)

Complexidade obrigatória: permitir $O(1)$ amortizado por requisição.

```
/**
 * Rate Limiter com algoritmo Sliding Window.
 * Cada cliente tem um Deque<Long> de timestamps das requisições.
 * Ao processar uma nova requisição, timestamps expirados são removidos
 * e o tamanho atual é comparado ao limite.
 *
 * Mais preciso que Fixed Window Counter (sem burst no limite da janela).
 */
@Component
public class RateLimiter {

    // ConcurrentHashMap garante thread-safety sem bloqueio global
    private final ConcurrentHashMap<String, Deque<Long>> janelas =
        new ConcurrentHashMap<>();

    // Configurável via application.properties
    @Value("${rate.limiter.max-requisicoes:100}")
    private int maxRequisicoes;

    @Value("${rate.limiter.janela-ms:60000}")
    private long janelaMilissegundos;

    /**
     * Verifica se o cliente pode fazer a requisição.
     *
     * @param clienteId identificador do cliente (IP, userId, API key)
     * @return true se dentro do limite; false se excedeu
     */
}
```

```

    */
    public boolean permitir(String clienteId) {
        long agora = System.currentTimeMillis();

        Deque<Long> timestamps = janelas.computeIfAbsent(
            clienteId, k -> new ArrayDeque<>());

        synchronized (timestamps) {
            // Remove timestamps fora da janela deslizando
            while (!timestamps.isEmpty() &&
                agora - timestamps.peekFirst() > janelaMilissegundos) {
                timestamps.pollFirst();
            }

            if (timestamps.size() >= maxRequisicoes) {
                return false; // limite excedido - 429 Too Many Requests
            }

            timestamps.addLast(agora);
            return true;
        }
    }

    /** Retorna estatísticas para o endpoint /admin/rate-limiter/stats */
    public Map<String, Object> stats(String clienteId) {
        Deque<Long> ts = janelas.getOrDefault(clienteId, new ArrayDeque<>());
        return Map.of(
            "cliente", clienteId,
            "requisicoes_usadas", ts.size(),
            "limite", maxRequisicoes,
            "janela_ms", janelaMilissegundos
        );
    }
}

// Filtro HTTP que aplica o Rate Limiter a todos os endpoints
@Component
public class RateLimitFilter extends OncePerRequestFilter {

    @Autowired private RateLimiter rateLimiter;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {

        String clienteId = extrairClienteId(request);

        if (!rateLimiter.permitir(clienteId)) {
            response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value()); // 429
            response.setContentType(MediaType.APPLICATION_JSON_VALUE);
            response.getWriter().write("""
                {"erro": "Rate limit excedido", "limite": 100, "janela": "60s"}
            """);
            return;
        }
        chain.doFilter(request, response);
    }
}

```

```
    }

    private String extrairClienteId(HttpServletRequest request) {
        // Tenta X-Forwarded-For (proxy/load balancer); cai no IP direto
        String forwarded = request.getHeader("X-Forwarded-For");
        return forwarded != null ? forwarded.split(",")[0].trim()
            : request.getRemoteAddr();
    }
}
```

5. Configuração Spring Boot

5.1 CacheConfig — Redis com TTL

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Value("${spring.cache.redis.time-to-live:300000}")
    private long ttlMs;

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration config = RedisCacheConfiguration
            .defaultCacheConfig()
            .entryTtl(Duration.ofMillis(ttlMs))
            .disableCachingNullValues()
            .serializeValuesWith(
                RedisSerializationContext.SerializationPair
                    .fromSerializer(new GenericJackson2JsonRedisSerializer())
            );

        return RedisCacheManager.builder(factory)
            .cacheDefaults(config)
            // Cache de desempenho com TTL diferenciado (dados mudam menos)
            .withCacheConfiguration("desempenhos",
                config.entryTtl(Duration.ofHours(1)))
            .build();
    }
}

// AlunoService - uso das anotações Spring Cache
@Service
public class AlunoService {

    @Autowired private AlunoRepository repository;
    @Autowired private AutocompletarService autocompletar;

    @Cacheable(value = "alunos", key = "#id")
    public AlunoDTO buscarPorId(Long id) {
        // Este método só é executado em cache MISS
        return repository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new AlunoNaoEncontradoException(id));
    }

    @CachePut(value = "alunos", key = "#result.id()")
    public AlunoDTO salvar(Aluno aluno) {
        Aluno salvo = repository.save(aluno);
        autocompletar.indexar(salvo.getNome()); // atualiza Trie
        return toDTO(salvo);
    }

    @CacheEvict(value = "alunos", key = "#id")
    public void remover(Long id) {
        repository.deleteById(id);
    }
}
```

```

@CacheEvict(value = "alunos", allEntries = true)
public void limparCache() { /* usado pelo endpoint /admin/cache */ }

private AlunoDTO toDTO(Aluno a) {
    return new AlunoDTO(a.getId(), a.getMatricula(), a.getNome(),
                        a.getEmail(), a.getCurso(), a.getPeriodo());
}
}

```

5.2 RabbitMQConfig e Mensageria

```

@Configuration
public class RabbitMQConfig {

    // Nomes das filas e exchanges - centralizados para evitar typos
    public static final String EXCHANGE_RELATORIOS = "edumetrics.relatorios";
    public static final String FILA_RELATORIOS =
"relatorios.processamento";
    public static final String FILA_DLQ = "relatorios.dlq";
    public static final String ROUTING_KEY_PDF = "relatorio.gerar";

    // — Exchange —————
    @Bean
    public TopicExchange exchangeRelatorios() {
        return ExchangeBuilder
            .topicExchange(EXCHANGE_RELATORIOS)
            .durable(true) // sobrevive a reinicializações
            .build();
    }

    // — Dead Letter Queue (DLQ) —————
    @Bean
    public Queue filaDLQ() {
        return QueueBuilder.durable(FILA_DLQ).build();
    }

    // — Fila principal com referência a DLQ —————
    @Bean
    public Queue filaRelatorios() {
        return QueueBuilder
            .durable(FILA_RELATORIOS)
            // Mensagens rejeitadas vão para a DLQ após 3 tentativas
            .withArgument("x-dead-letter-exchange", "")
            .withArgument("x-dead-letter-routing-key", FILA_DLQ)
            .build();
    }

    @Bean
    public Binding bindingRelatorios(Queue filaRelatorios,
                                     TopicExchange exchangeRelatorios) {
        return BindingBuilder
            .bind(filaRelatorios)
            .to(exchangeRelatorios)
            .with(ROUTING_KEY_PDF);
    }
}

```



```

@Bean
public Jackson2JsonMessageConverter messageConverter() {
    return new Jackson2JsonMessageConverter();
}

@Bean
public RabbitTemplate rabbitTemplate(ConnectionFactory cf,
    Jackson2JsonMessageConverter converter) {
    RabbitTemplate t = new RabbitTemplate(cf);
    t.setMessageConverter(converter);
    return t;
}

// Producer – retorna imediatamente sem esperar o processamento
@Service
public class RelatorioProducer {

    @Autowired private RabbitTemplate rabbitTemplate;

    public String solicitarRelatorio(Long alunoId, String tipo, String
semestre) {
        String correlationId = UUID.randomUUID().toString();
        var evento = new EventoRelatorio(
            correlationId, alunoId, tipo, semestre, Instant.now());

        rabbitTemplate.convertAndSend(
            RabbitMQConfig.EXCHANGE_RELATORIOS,
            RabbitMQConfig.ROUTING_KEY_PDF,
            evento
        );
        log.info("Relatório {} solicitado – correlationId={}", tipo,
correlationId);
        return correlationId;    // retorna ao controller; HTTP 202 Accepted
    }
}

// Consumer – processa em thread de background
@Component
public class RelatorioConsumer {

    @RabbitListener(queues = RabbitMQConfig.FILA_RELATORIOS)
    public void processar(EventoRelatorio evento) {
        log.info("Processando relatório {} para aluno {}",
            evento.tipo(), evento.alunoId());
        try {
            // Simula geração de PDF (substituir por iText/JasperReports em
produção)
            Thread.sleep(2_000);
            log.info("Relatório {} concluído – correlationId={}",
                evento.tipo(), evento.correlationId());
        } catch (Exception e) {
            log.error("Erro ao gerar relatório: {}", e.getMessage());
            // Lançar RuntimeException envia NACK → mensagem vai para DLQ
            throw new RuntimeException("Falha no processamento", e);
        }
    }
}

```

```

    }

    // Handler da DLQ – processa mensagens que falharam
    @RabbitListener(queues = RabbitMQConfig.FILA_DLQ)
    public void processarDLQ(EventoRelatorio evento) {
        log.error("Mensagem na DLQ – relatório {} para aluno {} não foi
gerado.",
            evento.tipo(), evento.alunoId());
        // Aqui: notificar monitoramento, salvar log de falha no BD, etc.
    }
}

```

5.3 RankingService com PriorityQueue

```

@Service
public class RankingService {

    @Autowired private DesempenhoRepository desempenhoRepo;

    /**
     * Retorna os top-K alunos por média geral.
     *
     * Algoritmo: min-heap de tamanho K.
     * Para cada aluno, insere no heap. Se heap.size() > K, remove o menor.
     * Ao final, o heap contém exatamente os K maiores.
     *
     * Complexidade:  $\Theta(n \log k)$  – n alunos, k posições no ranking.
     * Muito melhor que ordenar todos os n alunos:  $\Theta(n \log n)$ .
     *
     * @param k número de posições desejadas (ex: 10)
     */
    @Cacheable(value = "ranking", key = "#k")
    public List<RankingItemDTO> topK(int k) {
        if (k <= 0) throw new IllegalArgumentException("k deve ser positivo");

        // MIN-HEAP: o menor elemento fica na raiz
        // Isso nos permite remover o menor (menor média) quando o heap excede
        k

        PriorityQueue<RankingItemDTO> heap =
            new
PriorityQueue<>(Comparator.comparingDouble(RankingItemDTO::mediaGeral));

        desempenhoRepo.findMediasPorAluno().forEach(item -> {
            heap.offer(item);
            if (heap.size() > k) {
                heap.poll();    // remove o de MENOR média – mantém os maiores
            }
        });

        // Extrair em ordem decrescente (maior média primeiro)
        List<RankingItemDTO> ranking = new ArrayList<>(heap);

        ranking.sort(Comparator.comparingDouble(RankingItemDTO::mediaGeral).reversed());
        ;

        // Adicionar posição
    }
}

```

```
var resultado = new ArrayList<RankingItemDTO>();
for (int i = 0; i < ranking.size(); i++) {
    var item = ranking.get(i);
    resultado.add(new RankingItemDTO(
        i + 1,
        item.nome(), item.matricula(),
        item.mediaGeral(), item.disciplinasConcluidas()
    ));
}
return Collections.unmodifiableList(resultado);
}

// Invalidar ranking ao alterar nota
@CacheEvict(value = "ranking", allEntries = true)
public void invalidarRanking() { }
```

6. Especificação dos Endpoints REST

Todos os endpoints devem retornar JSON. Use `ResponseEntity<T>` nos controllers para controle explícito de status HTTP.

Método	Endpoint	Status	Descrição e comportamento esperado
POST	/api/alunos	201	Cadastrar aluno. Valida matrícula única. Insere nome na Trie.
GET	/api/alunos/{id}	200	@Cacheable("alunos"). Segundo acesso deve vir do Redis.
GET	/api/alunos/matricula/{mat}	200	Busca via LRUCache local — $O(1)$. Cache miss consulta o BD.
PUT	/api/alunos/{id}	200	@CachePut: atualiza BD e cache simultaneamente.
DELETE	/api/alunos/{id}	204	@CacheEvict: remove do BD e invalida cache.
GET	/api/disciplinas/autocompletar?q={q}	200	Trie.autocompletar(q, 10). $O(q)$ — instantâneo.
POST	/api/desempenhos	201	Registra nota. Invalida cache de ranking (@CacheEvict).
GET	/api/alunos/{id}/desempenho	200	@Cacheable("desempenhos"). Usa @EntityGraph para evitar N+1.
GET	/api/ranking?top={k}	200	PriorityQueue min-heap $O(n \log k)$. @Cacheable("ranking").
POST	/api/relatorios	202	Publica EventoRelatorio no RabbitMQ. Retorna correlationId.
GET	/api/admin/cache/stats	200	hits, misses, hitRate, entradas do LRUCache.
DELETE	/api/admin/cache	204	@CacheEvict(allEntries=true) em todos os caches.
GET	/api/admin/health	200	Status Redis, RabbitMQ e Trie (total de palavras indexadas).

6.1 Exemplo de Controller Completo

```
@RestController
@RequestMapping("/api/alunos")
@Validated
public class AlunoController {

    @Autowired private AlunoService alunoService;
    @Autowired private AutocompletarService autocompletar;

    @PostMapping
    public ResponseEntity<AlunoDTO> cadastrar(@Valid @RequestBody AlunoDTO dto)
    {
        Aluno aluno = new Aluno(/* mapear dto */);
        AlunoDTO salvo = alunoService.salvar(aluno);
        URI location = URI.create("/api/alunos/" + salvo.id());
        return ResponseEntity.created(location).body(salvo);
    }

    @GetMapping("/{id}")
    public ResponseEntity<AlunoDTO> buscar(@PathVariable Long id) {
        return ResponseEntity.ok(alunoService.buscarPorId(id));
    }

    @GetMapping("/matricula/{matricula}")
    public ResponseEntity<AlunoDTO> buscarPorMatricula(@PathVariable String
matricula) {
        return ResponseEntity.ok(alunoService.buscarPorMatricula(matricula));
    }

    @PutMapping("/{id}")
    public ResponseEntity<AlunoDTO> atualizar(
        @PathVariable Long id, @Valid @RequestBody AlunoDTO dto) {
        return ResponseEntity.ok(alunoService.atualizar(id, dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> remover(@PathVariable Long id) {
        alunoService.remover(id);
        return ResponseEntity.noContent().build();
    }
}

@RestController
@RequestMapping("/api/relatorios")
public class RelatorioController {

    @Autowired private RelatorioProducer producer;

    @PostMapping
    public ResponseEntity<Map<String, String>> solicitar(
        @RequestBody SolicitacaoRelatorioDTO req) {
        String correlationId = producer.solicitarRelatorio(
            req.alunoId(), req.tipo(), req.semestre());
        // HTTP 202 Accepted - não espera o processamento
    }
}
```

```
        return ResponseEntity.accepted().body(
            Map.of("correlationId", correlationId,
                "mensagem", "Relatorio em processamento. Use o correlationId
para rastrear."));
    }
}
```

7. Suíte de Benchmarking JMH

A suíte JMH é um dos entregáveis mais importantes do projeto e equivale a 20 pontos da rubrica. O grupo deve executar os benchmarks, registrar os resultados e apresentar uma análise qualitativa — não basta mostrar os números.

7.1 pom.xml — Dependências JMH

```
<dependencies>
  <!-- JMH Core -->
  <dependency>
    <groupId>org.openjdk.jmh</groupId>
    <artifactId>jmh-core</artifactId>
    <version>1.37</version>
  </dependency>
  <!-- Processador de anotações JMH (gera código em compile-time) -->
  <dependency>
    <groupId>org.openjdk.jmh</groupId>
    <artifactId>jmh-generator-annprocess</artifactId>
    <version>1.37</version>
    <scope>provided</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- Cria um uber-jar com todos os benchmarks -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <executions>
        <execution>
          <phase>package</phase>
          <goals><goal>shade</goal></goals>
          <configuration>
            <finalName>benchmarks</finalName>
            <transformers>
              <transformer
implementation="org.apache.maven.plugins.shade.resource.
ManifestResourceTransformer">
                <mainClass>org.openjdk.jmh.Main</mainClass>
              </transformer>
            </transformers>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

# Compilar e executar:
mvn clean package -DskipTests
java -jar target/benchmarks.jar

# Executar benchmark específico:
java -jar target/benchmarks.jar EduMetricsBenchmark.buscaMatricula

# Salvar resultado em arquivo:
```

```
java -jar target/benchmarks.jar -rf json -rff resultado.json
```

7.2 Classe EduMetricsBenchmark.java (Completa)

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MICROSECONDS)
@State(Scope.Benchmark)
@Warmup(iterations = 5, time = 1, timeUnit = TimeUnit.SECONDS)
@Measurement(iterations = 10, time = 1, timeUnit = TimeUnit.SECONDS)
@Fork(value = 2, jvmArgs = {"-Xms512m", "-Xmx1g", "-XX:+UseG1GC"})
public class EduMetricsBenchmark {

    private static final int N = 50_000;

    private Map<String, AlunoDTO> lruCache;
    private HashMap<String, AlunoDTO> hashMapIndice;
    private Trie trie;
    private RateLimiter rateLimiter;
    private PriorityQueue<RankingItemDTO> minHeap;

    private List<String> matriculas;
    private List<String> prefixos;

    @Setup(Level.Trial)
    public void setup() {
        lruCache = LRUCache.create(1000);
        hashMapIndice = new HashMap<>(N);
        trie = new Trie();
        rateLimiter = new RateLimiter();
        minHeap = new PriorityQueue<>(
            Comparator.comparingDouble(RankingItemDTO::mediaGeral));
        matriculas = new ArrayList<>(N);
        prefixos = List.of("Alg", "Est", "Pro", "Mat", "Ban");

        Random rand = new Random(42);
        String[] disciplinas = {
            "Algoritmos", "Estrutura de Dados", "Programação Web",
            "Matemática Discreta", "Banco de Dados", "Sistemas Operacionais"
        };

        for (int i = 0; i < N; i++) {
            String mat = String.format("2024%05d", i);
            var dto = new AlunoDTO((long)i, mat, "Aluno " + i,
                "aluno"+i+"@iftm.edu.br", "TSI", 5);
            matriculas.add(mat);
            hashMapIndice.put(mat, dto);
            lruCache.put(mat, dto);
            var ranking = new RankingItemDTO(i+1, "Aluno "+i, mat,
                5.0 + rand.nextDouble()*5.0, rand.nextInt(10)+1);
            minHeap.offer(ranking);
            if (minHeap.size() > 10) minHeap.poll(); // manter top-10
        }
        for (String d : disciplinas) trie.inserir(d);
    }

    // — Benchmark 1: LRU Cache vs HashMap direto —————
```



```

@Benchmark
public AlunoDTO buscaLRUCache(Blackhole bh) {
    String mat = matriculas.get(ThreadLocalRandom.current().nextInt(N));
    AlunoDTO dto = lruCache.get(mat);
    bh.consume(dto);
    return dto;
}

@Benchmark
public AlunoDTO buscaHashMapDireto(Blackhole bh) {
    String mat = matriculas.get(ThreadLocalRandom.current().nextInt(N));
    AlunoDTO dto = hashMapIndice.get(mat);
    bh.consume(dto);
    return dto;
}

// — Benchmark 2: Trie vs varredura linear —————
@Benchmark
public List<String> autocompletarTrie(Blackhole bh) {
    String pref =
prefixos.get(ThreadLocalRandom.current().nextInt(prefixos.size()));
    List<String> r = trie.autocompletar(pref, 10);
    bh.consume(r);
    return r;
}

@Benchmark
public List<String> autocompletarLinear(Blackhole bh) {
    String pref =
prefixos.get(ThreadLocalRandom.current().nextInt(prefixos.size())).toLowerCase(
);
    List<String> r = hashMapIndice.keySet().stream()
        .filter(k -> k.startsWith(pref)).limit(10).toList();
    bh.consume(r);
    return r;
}

// — Benchmark 3: Rate Limiter (Sliding Window) —————
@Benchmark
public boolean rateLimiterCheck(Blackhole bh) {
    // Simula 10 clientes distintos para não saturar um único
    String clienteId = "cliente-" +
        ThreadLocalRandom.current().nextInt(10);
    boolean ok = rateLimiter.permitir(clienteId);
    bh.consume(ok);
    return ok;
}
}

```

7.3 Template de Relatório JMH (Obrigatório no README)

O README deve conter uma tabela como a abaixo com os valores reais obtidos pelo grupo:

Benchmark	Score	Erro $\pm$	Unidade	Análise (mínimo 1 linha)
buscaLRUCache	—	—	$\mu\text{s/op}$	_____ _____
buscaHashMapDireto	—	—	$\mu\text{s/op}$	_____ _____
autocompletarTrie	—	—	$\mu\text{s/op}$	_____ _____
autocompletarLinear	—	—	$\mu\text{s/op}$	_____ _____
rateLimiterCheck	—	—	$\mu\text{s/op}$	_____ _____



O que analisar nos resultados JMH

1. LRU Cache vs HashMap: esperamos desempenho próximo (ambos $O(1)$), mas o LRU tem overhead da lista duplamente encadeada de ordenação. Quantas vezes mais lento?
2. Trie vs varredura linear: com 6 disciplinas, a diferença pode parecer pequena. Explique o que aconteceria com 10.000 disciplinas — qual a complexidade de cada?
3. Rate Limiter: o custo deve ser $O(1)$ amortizado. Se estiver acima de $1 \mu\text{s}$, o synchronized pode ser um gargalo. Discuta uma alternativa (StampedLock, etc.).
4. Variância (erro $\pm$): variância $> 10\%$ do score indica instabilidade — pode ser GC, JIT ainda aquecendo, ou carga do sistema. Justifique.

8. Suíte de Testes JUnit 5 (Cobertura Mínima: 80%)

8.1 Testes Obrigatórios por Componente

Classe de teste	Cenários obrigatórios
LRUCacheTest	armazenar e recuperar valor; respeitar capacidade máxima; manter elemento recém-acessado; remover o LRU correto em sequência de acessos; thread-safety (2 threads concorrentes)
TrieTest	inserir e recuperar por prefixo exato; prefixo inexistente retorna lista vazia; respeitar limite maxResultados; case-insensitive; inserir palavra duplicada não duplica no resultado
RateLimiterTest	permitir requisições dentro do limite; bloquear a (N+1)-ésima; limites independentes por cliente; após expirar a janela, permitir novamente; reset correto após janela
AlunoServiceTest	cache hit: segunda chamada não vai ao banco (verificar com Mockito.verify(times(1))); cache evict após remoção; exceção para aluno inexistente; insercao na Trie ao salvar
RankingServiceTest	Top-3 retorna os 3 com maior média; empate tratado sem exceção; k=1 retorna apenas o melhor; cache do ranking invalidado ao chamar invalidarRanking()
RelatorioProducerTest	publica mensagem no RabbitTemplate (mock); retorna correlationId não-nulo; tipo e alunold presentes no evento publicado

8.2 Exemplo de Teste Completo — LRUCacheTest

```
class LRUCacheTest {  
  
    private Map<String, Integer> cache;  
  
    @BeforeEach  
    void setUp() {  
        cache = LRUCache.create(3); // capacidade = 3  
    }  
  
    @Test  
    @DisplayName("Deve armazenar e recuperar um valor")  
    void deveArmazenarERecuperar() {  
        cache.put("a", 1);  
        assertEquals(1, cache.get("a"));  
    }  
  
    @Test  
    @DisplayName("Deve respeitar a capacidade máxima")
```

```

void deveRespeitarCapacidade() {
    cache.put("a", 1); cache.put("b", 2); cache.put("c", 3);
    cache.put("d", 4);    // "a" deve ser eviccionado (LRU)

    assertEquals(3, cache.size());
    assertNull(cache.get("a"),    "a deveria ter sido eviccionado");
    assertEquals(4, cache.get("d"));
}

@Test
@DisplayName("Deve preservar elemento recém-acessado via get()")
void devePreservarElementoAcessado() {
    cache.put("a", 1); cache.put("b", 2); cache.put("c", 3);
    cache.get("a");      // "a" torna-se o mais recente
    cache.put("d", 4);    // "b" deve ser eviccionado (agora o LRU)

    assertNotNull(cache.get("a"), "a foi acessado - nao deve ser
eviccionado");
    assertNull(cache.get("b"),    "b deve ter sido eviccionado");
}

@Test
@DisplayName("Deve ser thread-safe com acessos concorrentes")
void deveSerThreadSafe() throws InterruptedException {
    int threads = 20, ops = 1000;
    var executor = Executors.newFixedThreadPool(threads);
    var latch = new CountDownLatch(threads);

    for (int t = 0; t < threads; t++) {
        final int tid = t;
        executor.submit(() -> {
            for (int i = 0; i < ops; i++) {
                cache.put("t" + tid + "-k" + i, i);
                cache.get("t" + tid + "-k" + i);
            }
            latch.countDown();
        });
    }
    latch.await(10, TimeUnit.SECONDS);
    executor.shutdown();
    // Não deve lançar ConcurrentModificationException
    assertTrue(cache.size() <= 3, "Cache não deve exceder a capacidade");
}

// Gerando relatório JaCoCo:
// mvn clean test jacoco:report
// Resultado em: target/site/jacoco/index.html

```

9. Entregáveis e Checklist Final

9.1 Entregáveis Obrigatórios

#	Entregável	Critério de aceitação
1	Repositório GitHub	Público; histórico de commits com mensagens significativas; pelo menos 1 commit por integrante
2	Código Spring Boot	Compila com mvn clean package; inicia com docker compose up -d + mvn spring-boot:run sem erros
3	README.md	Instruções de execução; lista de endpoints; arquitetura descrita; tabela JMH preenchida com análise
4	Relatório JMH	Arquivo jmh-resultado.json (saída -rf json) + análise escrita no README ou PDF separado
5	Relatório JaCoCo	target/site/jacoco/index.html com cobertura $\geq 80\%$ nas classes domain e service
6	Apresentação oral	Demonstração ao vivo (sistema rodando); slides de arquitetura; resposta técnica às perguntas

9.2 Checklist de Auto-Avaliação

Verifique cada item antes da apresentação. Itens marcados com * São critérios de desclassificação.

✓ Infraestrutura e compilação

- ☐ docker compose up -d sobe Redis e RabbitMQ sem erros
- ☐ mvn clean package -DskipTests conclui sem erros *
- ☐ mvn spring-boot:run inicia a aplicação e conecta ao Redis e RabbitMQ
- ☐ H2 Console acessível em <http://localhost:8080/h2-console>
- ☐ Painel RabbitMQ acessível em <http://localhost:15672> (guest/guest)

✓ Estruturas customizadas

- ☐ LRUCache: testes de evicção e thread-safety passam *
- ☐ Trie: carregada no @PostConstruct; autocompletar funciona via </api/disciplinas/autocompletar>
- ☐ RateLimiter: retorna 429 após 100 requisições no mesmo IP em 1 minuto
- ☐ Nenhuma das 3 estruturas usa biblioteca externa para a lógica principal *

✓ Spring Boot e integração

- ☐ @Cacheable: segunda chamada a GET /api/alunos/{id} não atinge o banco (verificar no log)
- ☐ @CacheEvict: DELETE /api/alunos/{id} invalida o cache
- ☐ POST /api/relatorios retorna HTTP 202 em menos de 100 ms
- ☐ Consumer do RabbitMQ processa a mensagem e imprime log
- ☐ DLQ configurada: mensagem com exceção vai para relatorios.dlq (verificar no painel)
- ☐ N+1 queries ausentes: spring.jpa.show-sql=true mostra <= 2 queries por endpoint *

✓ Testes e benchmarking

- ☐ mvn test: todos os testes passam *
- ☐ Cobertura JaCoCo >= 80% nos pacotes domain e service *
- ☐ Suíte JMH com os 3 benchmarks executada e resultado salvo *
- ☐ Relatório JMH inclui análise qualitativa (não apenas números)
- ☐ README contém tabela de resultados JMH preenchida

✓ Qualidade e documentação

- ☐ Pacotes organizados conforme estrutura definida na Seção 2.4
- ☐ Nomenclatura em português BR consistente (exceto termos técnicos)
- ☐ README.md completo (instrucoes + arquitetura + JMH)
- ☐ Cada integrante consegue explicar qualquer parte do código
- ☐ Histórico de commits mostra contribuição de todos os integrantes

10. Referências

1. BLOCH, J. Effective Java. 3. ed. Boston: Addison-Wesley, 2018. (Itens 10-12: hashCode, equals; Item 47: Streams vs iterables).
2. CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022. (Cap. 11: Hash Tables; Cap. 6: Heapsort e filas de prioridade).
3. ORACLE CORPORATION. JMH: Java Microbenchmark Harness. Disponível em: <https://openjdk.org/projects/code-tools/jmh/>. Acesso em: maio 2026.
4. SPRING FRAMEWORK. Cache Abstraction Reference Documentation. Disponível em: <https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#cache>. Acesso em: maio 2026.
5. REDIS LTD. Redis Documentation. Disponível em: <https://redis.io/docs/>. Acesso em: maio 2026.
6. RABBITMQ TEAM. Spring AMQP Reference. Disponível em: <https://www.rabbitmq.com/documentation.html>. Acesso em: maio 2026.
7. BAUER, C.; KING, G.; GREGORY, G. Java Persistence with Hibernate. 2. ed. Shelter Island: Manning, 2015. (Cap. 12: Fetching e o problema N+1).
8. KNUTH, D. E. The Art of Computer Programming, Vol. 3: Sorting and Searching. 2. ed. Boston: Addison-Wesley, 1998. (Cap. 6.3: Digital Searching — Tries).